```
-- ============================================================================
-- ASCEND - GUIA DE CONSULTAS SQL PARA LA DEFENSA
-- Base de datos: ascend (MySQL 8)
-- ============================================================================
-- Objetivo:
--   Reunir las consultas de lectura que la aplicacion ejecuta en sus modelos
--   y las consultas adicionales que pueden pedirse en una defensa oral.
--   Las consultas van de lo simple a lo complejo y se pueden ejecutar tal cual
--   en phpMyAdmin, MySQL Workbench o la consola mysql.
--
-- Como usar:
--   1. Elegir la base ascend (o descomentar USE ascend;).
--   2. Reemplazar los IDs de ejemplo por los que correspondan.
--   3. Los IDs de ejemplo pertenecen a la semilla QA:
--        administradores: 1 y 1003
--        organizadores:   2, 1000, 1001, 1002, 1004, 1005
--        jugadores:       1010 en adelante
--        equipos:         1100 en adelante
--        torneos:         1200 en adelante
--
-- Valores frecuentes:
--   usuarios.rol_id:        1 Administrador General, 2 Organizador,
--                           3 Participante/Jugador, 4 Usuario Publico
--   usuarios.esta_activo:   1 activo, 0 baja logica
--   equipos.activo:         1 activo, 0 baja logica
--   torneos.estado:         borrador, inscripciones_abiertas, en_curso,
--                           finalizado, cancelado
--   participantes_torneo.estado: pendiente, confirmado, rechazado, cancelado
--   torneo_encuentros.estado:    programado, en_curso, finalizado,
--                                cancelado, aplazado
--
-- Convencion de comentarios:
--   Modelo: metodo del archivo codigo_fuente/modelos/*.php de donde sale la
--           consulta (adaptada para ejecutarse con valores literales).
--   Extra:  consulta adicional de utilidad para la defensa.
-- ============================================================================

-- USE ascend;

-- ============================================================================
-- NIVEL 1 - CONSULTAS DE UNA SOLA TABLA O CON UN JOIN DIRECTO
-- ============================================================================

-- Q01. Cuantos usuarios hay registrados por rol.
-- Modelo: Usuario::contarTotalJugadoresRegistrados(),
contarTotalOrganizadoresRegistrados()
SELECT r.id AS rol_id,
       r.nombre_rol,
       COUNT(u.id) AS total_usuarios
FROM roles r
LEFT JOIN usuarios u ON u.rol_id = r.id
GROUP BY r.id, r.nombre_rol
```

```sql
ORDER BY r.id;

-- Q02. Listar todos los organizadores registrados.
-- Modelo: Usuario::obtenerTodosUsuariosConRoles(2) + Organizador::obtenerPerfil()
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       u.esta_activo,
       u.email_verificado,
       u.fecha_registro,
       COALESCE(NULLIF(po.nombre_organizacion, ''), 'Organizador Independiente')
AS organizacion,
       COALESCE(po.verificado_oficial, 0) AS verificado_oficial,
       po.localidad,
       po.sitio_web
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.rol_id = 2
ORDER BY u.nombre_completo;

-- Q03. Listar todos los jugadores registrados.
-- Modelo: Usuario::obtenerTodosUsuariosConRoles(3)
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.esta_activo,
       u.email_verificado,
       u.fecha_registro,
       pj.apodo_gamertag,
       pj.nivel,
       pj.pais,
       pj.ciudad
FROM usuarios u
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
WHERE u.rol_id = 3
ORDER BY u.nombre_completo;

-- Q04. Cantidad de cuentas activas e inactivas por rol.
-- Extra
SELECT r.nombre_rol,
       SUM(u.esta_activo = 1) AS activos,
       SUM(u.esta_activo = 0) AS inactivos,
       COUNT(*) AS total
FROM usuarios u
INNER JOIN roles r ON r.id = u.rol_id
GROUP BY r.id, r.nombre_rol
ORDER BY r.id;

-- Q05. Ultimos jugadores registrados.
-- Modelo: Usuario::obtenerUltimosJugadoresRegistrados()
SELECT id, nombre_completo, email, fecha_registro
FROM usuarios
WHERE rol_id = 3
```

```sql
ORDER BY fecha_registro DESC
LIMIT 10;

-- Q06. Ultimos organizadores registrados.
-- Modelo: Usuario::obtenerUltimosOrganizadoresRegistrados()
SELECT id, nombre_completo, email, fecha_registro
FROM usuarios
WHERE rol_id = 2
ORDER BY fecha_registro DESC
LIMIT 10;

-- Q07. Buscar un usuario por correo electronico.
-- Modelo: Usuario::buscarPorEmail()
SELECT id, nombre_completo, email, rol_id, esta_activo, email_verificado,
fecha_registro
FROM usuarios
WHERE email = 'admin@ascend.com'
LIMIT 1;

-- Q08. Cuentas activas con correo sin verificar (riesgo de seguridad).
-- Extra
SELECT id, nombre_completo, email, rol_id, fecha_registro
FROM usuarios
WHERE esta_activo = 1
  AND email_verificado = 0
ORDER BY fecha_registro DESC;

-- Q09. Juegos activos del catalogo.
-- Modelo: Juego::obtenerJuegosActivos()
SELECT id, nombre, categoria, puntos_victoria, puntos_empate, puntos_derrota
FROM juegos
WHERE activo = 1
ORDER BY nombre;

-- Q10. Formatos de torneo activos.
-- Modelo: Formato::obtenerActivas()
SELECT codigo, nombre, descripcion
FROM formatos_torneo
WHERE activo = 1
ORDER BY codigo;

-- Q11. Modalidades y cantidad de juegos que las permiten.
-- Modelo: Modalidad::obtenerTodas() + juego_modalidades_permitidas
SELECT m.id,
       m.nombre,
       COUNT(jmp.juego_id) AS juegos_que_la_permiten
FROM modalidades m
LEFT JOIN juego_modalidades_permitidas jmp ON jmp.modalidad_id = m.id
GROUP BY m.id, m.nombre
ORDER BY m.id;

-- Q12. Sistemas de puntuacion disponibles.
-- Modelo: Torneo::obtenerSistemasPuntuacion()
SELECT id, nombre, puntos_victoria, puntos_empate, puntos_derrota, descripcion
```

```sql
FROM sistemas_puntuacion
ORDER BY nombre;

-- Q13. Juegos que permiten modalidad de equipos.
-- Modelo: Juego::obtenerJuegosParaEquipos()
SELECT j.id, j.nombre, j.categoria
FROM juegos j
INNER JOIN juego_modalidades_permitidas jmp ON jmp.juego_id = j.id
WHERE j.activo = 1
  AND jmp.modalidad_id = 2
ORDER BY j.nombre;

-- Q14. Juegos publicos con su primera modalidad permitida.
-- Modelo: Juego::obtenerJuegosPublicos()
SELECT j.id,
       j.nombre,
       j.categoria,
       j.imagen_url,
       (SELECT m.nombre
        FROM juego_modalidades_permitidas jmp
        INNER JOIN modalidades m ON m.id = jmp.modalidad_id
        WHERE jmp.juego_id = j.id
        ORDER BY m.id
        LIMIT 1) AS modalidad
FROM juegos j
WHERE j.activo = 1
ORDER BY j.nombre;

-- ============================================================================
-- NIVEL 2 - PERFILES, LOGROS, GUARDADOS Y NOTIFICACIONES
-- ============================================================================

-- Q15. Perfil completo de un jugador (cuenta + datos deportivos).
-- Modelo: Jugador::obtenerPerfil()
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       u.foto_perfil_url,
       pj.apodo_gamertag,
       pj.bio,
       pj.banner_url,
       pj.nivel,
       pj.experiencia_puntos,
       pj.pais,
       pj.ciudad,
       pj.fecha_nacimiento,
       pj.discord_tag,
       pj.instagram_url,
       pj.twitter_url,
       pj.campos_publicos
FROM usuarios u
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
WHERE u.id = 1010
```

```sql
LIMIT 1;

-- Q16. Perfil publico de un jugador respetando campos_publicos.
-- Modelo: Jugador::obtenerPerfilPublico() + listaCamposPublicos()
SELECT u.id,
       u.nombre_completo,
       pj.apodo_gamertag,
       pj.nivel,
       pj.experiencia_puntos,
       CASE WHEN FIND_IN_SET('bio', pj.campos_publicos) THEN pj.bio END AS
bio_publica,
       CASE WHEN FIND_IN_SET('ubicacion', pj.campos_publicos)
            THEN CONCAT_WS(', ', NULLIF(pj.ciudad, ''), NULLIF(pj.pais, '')) END
AS ubicacion_publica,
       CASE WHEN FIND_IN_SET('edad', pj.campos_publicos) THEN pj.fecha_nacimiento
END AS nacimiento_publico,
       CASE WHEN FIND_IN_SET('discord', pj.campos_publicos) THEN pj.discord_tag
END AS discord_publico,
       CASE WHEN FIND_IN_SET('instagram', pj.campos_publicos) THEN
pj.instagram_url END AS instagram_publico,
       CASE WHEN FIND_IN_SET('twitter', pj.campos_publicos) THEN pj.twitter_url
END AS twitter_publico
FROM usuarios u
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
WHERE u.id = 1010
  AND u.rol_id = 3
  AND u.esta_activo = 1
LIMIT 1;

-- Q17. Perfil completo de un organizador.
-- Modelo: Organizador::obtenerPerfil()
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       u.foto_perfil_url,
       po.nombre_organizacion,
       po.bio_organizacion,
       po.localidad,
       po.telefono_contacto,
       po.sitio_web,
       po.verificado_oficial,
       po.banner_url,
       po.campos_publicos
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.id = 1000
  AND u.rol_id = 2
LIMIT 1;

-- Q18. Perfil publico de un organizador activo.
-- Modelo: Organizador::obtenerPerfilPublico()
SELECT u.id,
       u.nombre_completo,
```

```
       u.foto_perfil_url,
       po.nombre_organizacion,
       po.bio_organizacion,
       po.localidad,
       po.telefono_contacto,
       po.sitio_web,
       po.verificado_oficial,
       po.banner_url,
       po.campos_publicos
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.id = 1000
  AND u.rol_id = 2
  AND u.esta_activo = 1
LIMIT 1;

-- Q19. Organizadores sin fila en perfiles_organizadores.
-- Extra (control de integridad de la herencia 1:1)
SELECT u.id, u.nombre_completo, u.email, u.fecha_registro
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.rol_id = 2
  AND po.usuario_id IS NULL
ORDER BY u.id;

-- Q20. Jugadores sin fila en perfiles_jugadores.
-- Extra (control de integridad de la herencia 1:1)
SELECT u.id, u.nombre_completo, u.email, u.fecha_registro
FROM usuarios u
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
WHERE u.rol_id = 3
  AND pj.usuario_id IS NULL
ORDER BY u.id;

-- Q21. Logros de un jugador, primero desbloqueados.
-- Modelo: Jugador::obtenerLogros()
SELECT l.nombre,
       l.descripcion,
       l.icono,
       ul.estado,
       ul.progreso_actual,
       ul.progreso_objetivo
FROM usuario_logros ul
INNER JOIN logros l ON l.id = ul.logro_id
WHERE ul.usuario_id = 1010
ORDER BY FIELD(ul.estado, 'desbloqueado', 'en_curso', 'bloqueado'), l.id;

-- Q22. Equipos y torneos guardados por un usuario (asociacion polimorfica).
-- Modelo: Guardado::obtenerEquiposGuardados() y obtenerTorneosGuardados()
SELECT g.tipo,
       g.referencia_id,
       g.fecha_guardado,
       COALESCE(e.nombre_equipo, t.nombre) AS nombre
FROM guardados g
```

```sql
LEFT JOIN equipos e ON g.tipo = 'equipo' AND e.id = g.referencia_id
LEFT JOIN torneos t ON g.tipo = 'torneo' AND t.id = g.referencia_id
WHERE g.usuario_id = 1010
ORDER BY g.fecha_guardado DESC;

-- Q23. Ultimas notificaciones de un usuario.
-- Modelo: Notificacion::obtenerRecientes()
SELECT id, tipo, titulo, mensaje, leido, creado_en
FROM notificaciones
WHERE usuario_id = 1010
ORDER BY creado_en DESC, id DESC
LIMIT 20;

-- Q24. Cantidad de notificaciones no leidas de un usuario.
-- Modelo: Notificacion::contarNoLeidas()
SELECT COUNT(*) AS no_leidas
FROM notificaciones
WHERE usuario_id = 1010
  AND leido = 0;

-- Q25. Notificaciones no leidas agrupadas por usuario.
-- Extra (panel de administracion)
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(n.id) AS no_leidas
FROM usuarios u
INNER JOIN notificaciones n ON n.usuario_id = u.id AND n.leido = 0
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY no_leidas DESC, u.nombre_completo;

-- ============================================================================
-- NIVEL 3 - EQUIPOS, MIEMBROS, CAPITANES Y SOLICITUDES
-- ============================================================================

-- Q26. Ficha publica de un equipo con su juego.
-- Modelo: Equipo::obtenerPorId()
SELECT e.*,
       j.nombre AS nombre_juego,
       j.imagen_url AS juego_icono
FROM equipos e
LEFT JOIN juegos j ON j.id = e.juego_id
WHERE e.id = 1100
  AND e.activo = 1
LIMIT 1;

-- Q27. Equipos recientes con cantidad de miembros.
-- Modelo: Equipo::obtenerUltimosEquiposCreados()
SELECT e.id,
       e.nombre_equipo,
       e.creado_por,
       COUNT(em.usuario_id) AS total_miembros
FROM equipos e
LEFT JOIN equipo_miembros em ON em.equipo_id = e.id
```

```sql
GROUP BY e.id, e.nombre_equipo, e.creado_por
ORDER BY e.id DESC
LIMIT 10;

-- Q28. Directorio de equipos con creador, miembros activos y capitan principal.
-- Modelo: Equipo::obtenerDirectorioAdmin() (version general)
SELECT e.id,
       e.nombre_equipo,
       e.activo,
       e.fecha_creacion,
       j.nombre AS juego,
       u.nombre_completo AS creador,
       (SELECT COUNT(*)
        FROM equipo_miembros em
        WHERE em.equipo_id = e.id AND em.es_activo = 1) AS miembros_activos,
       (SELECT COUNT(*)
        FROM equipo_capitanes ec
        WHERE ec.equipo_id = e.id) AS capitanes,
       (SELECT u2.nombre_completo
        FROM equipo_capitanes ec2
        INNER JOIN usuarios u2 ON u2.id = ec2.usuario_id
        WHERE ec2.equipo_id = e.id
        ORDER BY ec2.es_capitan_principal DESC, ec2.fecha_asignacion ASC
        LIMIT 1) AS capitan_principal
FROM equipos e
LEFT JOIN juegos j ON j.id = e.juego_id
INNER JOIN usuarios u ON u.id = e.creado_por
ORDER BY e.fecha_creacion DESC;

-- Q29. Miembros activos de un equipo.
-- Modelo: Equipo::obtenerMiembros()
SELECT u.id,
       u.nombre_completo,
       u.foto_perfil_url,
       em.posicion,
       em.numero_camiseta,
       em.fecha_union
FROM equipo_miembros em
INNER JOIN usuarios u ON u.id = em.usuario_id
WHERE em.equipo_id = 1100
  AND em.es_activo = 1
ORDER BY u.nombre_completo;

-- Q30. Capitanes de un equipo, primero el principal.
-- Modelo: Equipo::obtenerCapitanes()
SELECT u.id,
       u.nombre_completo,
       u.foto_perfil_url,
       ec.es_capitan_principal,
       ec.fecha_asignacion,
       asignador.nombre_completo AS asignado_por
FROM equipo_capitanes ec
INNER JOIN usuarios u ON u.id = ec.usuario_id
INNER JOIN usuarios asignador ON asignador.id = ec.asignado_por
```

```sql
WHERE ec.equipo_id = 1100
ORDER BY ec.es_capitan_principal DESC;

-- Q31. Equipos de un jugador e indicador de capitan.
-- Modelo: Equipo::obtenerEquiposDelJugador()
SELECT e.id,
       e.nombre_equipo,
       e.juego_id,
       e.escudo_url,
       e.descripcion,
       em.es_activo AS membresia_activa,
       EXISTS (
           SELECT 1
           FROM equipo_capitanes ec
           WHERE ec.equipo_id = e.id
             AND ec.usuario_id = em.usuario_id
       ) AS es_capitan
FROM equipos e
INNER JOIN equipo_miembros em ON em.equipo_id = e.id
WHERE em.usuario_id = 1010
  AND em.es_activo = 1
  AND e.activo = 1
ORDER BY e.nombre_equipo;

-- Q32. Equipos creados por usuarios que hoy son organizadores o administradores.
-- Extra (auditoria de datos: el flujo real de la aplicacion solo permite crear
-- equipos al rol Jugador; esta consulta detecta casos historicos o de semilla)
SELECT u.id AS creador_id,
       u.nombre_completo,
       r.nombre_rol,
       COUNT(e.id) AS equipos_creados,
       GROUP_CONCAT(e.nombre_equipo ORDER BY e.id SEPARATOR ' | ') AS equipos
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
INNER JOIN roles r ON r.id = u.rol_id
WHERE u.rol_id IN (1, 2)
GROUP BY u.id, u.nombre_completo, r.nombre_rol
ORDER BY equipos_creados DESC;

-- Q33. Cantidad de equipos activos por juego.
-- Extra
SELECT j.id,
       j.nombre AS juego,
       COUNT(e.id) AS equipos_activos
FROM juegos j
LEFT JOIN equipos e ON e.juego_id = j.id AND e.activo = 1
GROUP BY j.id, j.nombre
ORDER BY equipos_activos DESC, j.nombre;

-- Q34. Equipos sin miembros activos.
-- Extra (control de integridad)
SELECT e.id, e.nombre_equipo, e.fecha_creacion, e.activo
FROM equipos e
LEFT JOIN equipo_miembros em ON em.equipo_id = e.id AND em.es_activo = 1
```

```sql
WHERE em.usuario_id IS NULL
ORDER BY e.id;

-- Q35. Equipos sin capitan asignado.
-- Extra (control de integridad)
SELECT e.id, e.nombre_equipo, e.fecha_creacion
FROM equipos e
LEFT JOIN equipo_capitanes ec ON ec.equipo_id = e.id
WHERE ec.equipo_id IS NULL
ORDER BY e.id;

-- Q36. Equipos con mas de un capitan principal.
-- Extra (control de integridad)
SELECT ec.equipo_id,
       e.nombre_equipo,
       COUNT(*) AS capitanes_principales
FROM equipo_capitanes ec
INNER JOIN equipos e ON e.id = ec.equipo_id
WHERE ec.es_capitan_principal = 1
GROUP BY ec.equipo_id, e.nombre_equipo
HAVING COUNT(*) > 1;

-- Q37. Capitanes que no son miembros activos del equipo.
-- Extra (control de integridad)
SELECT ec.equipo_id,
       e.nombre_equipo,
       u.id AS capitan_id,
       u.nombre_completo AS capitan,
       ec.es_capitan_principal,
       em.es_activo AS membresia_activa
FROM equipo_capitanes ec
INNER JOIN equipos e ON e.id = ec.equipo_id
INNER JOIN usuarios u ON u.id = ec.usuario_id
LEFT JOIN equipo_miembros em ON em.equipo_id = ec.equipo_id AND em.usuario_id =
ec.usuario_id
WHERE em.usuario_id IS NULL
   OR em.es_activo = 0
ORDER BY ec.equipo_id;

-- Q38. Solicitudes de ingreso pendientes de un equipo.
-- Modelo: Equipo::obtenerSolicitudesPendientesDelEquipo()
SELECT s.id,
       s.mensaje,
       s.fecha_solicitud,
       u.id AS usuario_id,
       u.nombre_completo,
       u.foto_perfil_url
FROM solicitudes_equipo s
INNER JOIN usuarios u ON u.id = s.usuario_id
WHERE s.equipo_id = 1100
  AND s.estado = 'pendiente'
  AND s.iniciado_por = s.usuario_id
ORDER BY s.fecha_solicitud ASC;
```

```sql
-- Q39. Invitaciones recibidas por un jugador.
-- Modelo: Equipo::obtenerInvitacionesRecibidas()
SELECT s.id,
       s.mensaje,
       s.fecha_solicitud,
       e.id AS equipo_id,
       e.nombre_equipo,
       e.escudo_url,
       u.nombre_completo AS invitado_por_nombre
FROM solicitudes_equipo s
INNER JOIN equipos e ON e.id = s.equipo_id
INNER JOIN usuarios u ON u.id = s.iniciado_por
WHERE s.usuario_id = 1010
  AND s.estado = 'pendiente'
  AND s.iniciado_por <> s.usuario_id
ORDER BY s.fecha_solicitud DESC;

-- Q40. Solicitudes enviadas por un jugador.
-- Modelo: Equipo::obtenerSolicitudesEnviadas()
SELECT s.id,
       s.estado,
       s.fecha_solicitud,
       s.fecha_respuesta,
       e.id AS equipo_id,
       e.nombre_equipo,
       e.escudo_url
FROM solicitudes_equipo s
INNER JOIN equipos e ON e.id = s.equipo_id
WHERE s.usuario_id = 1010
  AND s.iniciado_por = s.usuario_id
ORDER BY s.fecha_solicitud DESC;

-- Q41. Jugadores que se pueden invitar a un equipo.
-- Modelo: Equipo::buscarJugadoresParaInvitar()
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.foto_perfil_url,
       pj.apodo_gamertag
FROM usuarios u
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
WHERE u.rol_id = 3
  AND u.esta_activo = 1
  AND (u.nombre_completo LIKE '%mateo%' OR pj.apodo_gamertag LIKE '%mateo%')
  AND u.id NOT IN (
      SELECT usuario_id
      FROM equipo_miembros
      WHERE equipo_id = 1100 AND es_activo = 1
  )
  AND u.id NOT IN (
      SELECT usuario_id
      FROM solicitudes_equipo
      WHERE equipo_id = 1100 AND estado = 'pendiente'
  )
```

```sql
ORDER BY u.nombre_completo
LIMIT 20;

-- Q42. Torneos en los que participa un equipo (cualquier estado de inscripcion).
-- Modelo: Equipo::obtenerTorneosDelEquipo()
SELECT pt.id AS participacion_id,
       pt.nombre AS nombre_participacion,
       pt.estado AS estado_inscripcion,
       pt.fecha_inscripcion,
       t.id AS torneo_id,
       t.nombre AS torneo_nombre,
       t.formato,
       t.estado AS torneo_estado,
       t.fecha_inicio,
       t.fecha_fin,
       t.banner_url
FROM participantes_torneo pt
INNER JOIN torneos t ON t.id = pt.torneo_id
WHERE pt.tipo = 'equipo'
  AND pt.referencia_id = 1100
ORDER BY t.fecha_inicio IS NULL, t.fecha_inicio DESC, t.id DESC;

-- Q43. Posicion de un equipo en cada torneo (version con ventana).
-- Modelo: Equipo::obtenerPosicionesDelEquipo() usa una subconsulta
correlacionada;
-- RANK() produce la misma clasificacion por puntos, diferencia y puntos a favor.
WITH clasificacion AS (
    SELECT pos.torneo_id,
           pos.participante_id,
           pos.puntos,
           pos.diferencia_goles,
           pos.puntos_favor,
           RANK() OVER (
               PARTITION BY pos.torneo_id
               ORDER BY pos.puntos DESC, pos.diferencia_goles DESC,
pos.puntos_favor DESC
           ) AS posicion
    FROM torneo_posiciones pos
)
SELECT t.id AS torneo_id,
       t.nombre AS torneo,
       t.estado AS torneo_estado,
       c.posicion,
       c.puntos,
       c.diferencia_goles,
       c.puntos_favor
FROM equipos e
INNER JOIN participantes_torneo p
        ON p.tipo = 'equipo'
       AND p.referencia_id = e.id
       AND p.estado = 'confirmado'
INNER JOIN torneos t ON t.id = p.torneo_id
INNER JOIN clasificacion c
        ON c.torneo_id = p.torneo_id
```

```sql
       AND c.participante_id = p.id
WHERE e.id = 1100
ORDER BY t.fecha_inicio DESC, t.id DESC;

-- Q44. Partidos de un equipo con resultado y condicion local/visitante.
-- Extra (adaptacion de Equipo::obtenerPartidosAdmin())
SELECT enc.id,
       enc.torneo_id,
       t.nombre AS torneo,
       enc.ronda,
       enc.fecha_hora_programada,
       enc.estado,
       enc.resultado_local,
       enc.resultado_visitante,
       IF(enc.participante_local_id = pt.id, 'Local', 'Visitante') AS condicion,
       COALESCE(rival.nombre, 'Sin rival') AS rival,
       CASE
           WHEN enc.estado = 'finalizado' AND enc.participante_ganador_id = pt.id
THEN 'V'
           WHEN enc.estado = 'finalizado' AND enc.participante_ganador_id IS NULL
THEN 'E'
           WHEN enc.estado = 'finalizado' THEN 'D'
           ELSE 'P'
       END AS resultado
FROM torneo_encuentros enc
INNER JOIN torneos t ON t.id = enc.torneo_id
INNER JOIN participantes_torneo pt
        ON pt.torneo_id = enc.torneo_id
       AND pt.tipo = 'equipo'
       AND pt.referencia_id = 1100
       AND (enc.participante_local_id = pt.id OR enc.participante_visitante_id =
pt.id)
LEFT JOIN participantes_torneo rival
       ON rival.id = IF(enc.participante_local_id = pt.id,
                        enc.participante_visitante_id,
                        enc.participante_local_id)
ORDER BY COALESCE(enc.fecha_hora_programada, enc.creado_en) DESC, enc.id DESC
LIMIT 30;

-- Q45. Resumen deportivo de un equipo: torneos, lider y campeon.
-- Modelo: Equipo::obtenerEstadisticasTorneos()
SELECT pt.id AS participacion_id,
       pt.torneo_id,
       pt.nombre AS participacion,
       t.estado AS torneo_estado,
       t.formato AS torneo_formato,
       (SELECT valor FROM torneo_config
        WHERE torneo_id = t.id AND clave = 'podio') AS podio,
       (SELECT participante_id
        FROM torneo_posiciones
        WHERE torneo_id = t.id
        ORDER BY puntos DESC, diferencia_goles DESC, puntos_favor DESC
        LIMIT 1) AS lider_torneo_id,
       (SELECT participante_ganador_id
```

```
        FROM torneo_encuentros
        WHERE torneo_id = t.id AND estado = 'finalizado'
        ORDER BY ronda DESC, id DESC
        LIMIT 1) AS campeon_torneo_id
FROM participantes_torneo pt
INNER JOIN torneos t ON t.id = pt.torneo_id
WHERE pt.tipo = 'equipo'
  AND pt.referencia_id = 1100;

-- Q46. Buscar un equipo por codigo de invitacion.
-- Modelo: Equipo::unirsePorCodigo()
SELECT id, nombre_equipo, codigo_invitacion
FROM equipos
WHERE codigo_invitacion = 'CODIGO-DE-EJEMPLO'
  AND activo = 1
LIMIT 1;

-- ============================================================================
-- NIVEL 4 - TORNEOS, INSCRIPCIONES, FIXTURE Y RESULTADOS
-- ============================================================================

-- Q47. Catalogo publico de torneos (excluye borrador y cancelado).
-- Modelo: Torneo::obtenerTorneosPublicados()
SELECT t.id,
       t.nombre,
       t.formato,
       t.estado,
       t.cupo_max_equipos,
       t.fecha_inicio,
       t.fecha_fin,
       t.banner_url,
       t.localidad,
       t.ubicacion,
       t.comunidad,
       t.juego_id,
       j.nombre AS nombre_juego,
       j.categoria AS categoria_juego,
       m.nombre AS nombre_modalidad
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
INNER JOIN modalidades m ON m.id = t.modalidad_id
WHERE t.estado <> 'borrador'
  AND t.estado <> 'cancelado'
ORDER BY t.fecha_inicio IS NULL, t.fecha_inicio ASC;

-- Q48. Torneos destacados (inscripciones abiertas o en curso).
-- Modelo: Torneo::obtenerTorneosDestacados()
SELECT t.id,
       t.nombre,
       t.formato,
       t.estado,
       t.cupo_max_equipos,
       t.fecha_inicio,
       t.fecha_fin,
```

```
       t.banner_url,
       j.nombre AS nombre_juego,
       j.categoria AS categoria_juego
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
WHERE t.estado IN ('inscripciones_abiertas', 'en_curso')
  AND (t.estado <> 'inscripciones_abiertas'
       OR t.fecha_limite_inscripcion IS NULL
       OR t.fecha_limite_inscripcion >= CURDATE())
ORDER BY (t.estado = 'inscripciones_abiertas') DESC,
         t.fecha_inicio IS NULL,
         t.fecha_inicio ASC
LIMIT 10;

-- Q49. Detalle publico de un torneo.
-- Modelo: Torneo::obtenerTorneoPublico()
SELECT t.*,
       j.nombre AS nombre_juego,
       j.categoria AS categoria_juego,
       m.nombre AS nombre_modalidad,
       u.nombre_completo AS nombre_organizador,
       u.foto_perfil_url AS foto_organizador
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
INNER JOIN modalidades m ON m.id = t.modalidad_id
INNER JOIN usuarios u ON u.id = t.organizador_id
LEFT JOIN perfiles_organizadores po ON po.usuario_id = t.organizador_id
WHERE t.id = 1200
  AND t.estado <> 'borrador'
  AND t.estado <> 'cancelado'
LIMIT 1;

-- Q50. Torneos de un estado determinado.
-- Extra (cambiar 'inscripciones_abiertas' por 'finalizado', 'borrador', etc.)
SELECT t.id,
       t.nombre,
       t.estado,
       t.fecha_inicio,
       t.fecha_fin,
       j.nombre AS juego,
       u.nombre_completo AS organizador
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
INNER JOIN usuarios u ON u.id = t.organizador_id
WHERE t.estado = 'inscripciones_abiertas'
ORDER BY t.fecha_inicio IS NULL, t.fecha_inicio ASC;

-- Q51. Torneos publicos de un organizador.
-- Modelo: Organizador::obtenerTorneosPublicos()
SELECT t.id,
       t.nombre,
       t.estado,
       t.banner_url,
       t.fecha_inicio,
```

```
       j.nombre AS nombre_juego
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
WHERE t.organizador_id = 1000
  AND t.estado NOT IN ('borrador', 'cancelado')
ORDER BY t.id DESC
LIMIT 10;

-- Q52. Participantes confirmados de un torneo (equipos o jugadores).
-- Modelo: Torneo::obtenerParticipantesPublicos()
SELECT p.id,
       p.tipo,
       p.referencia_id,
       p.nombre,
       CASE WHEN p.tipo = 'equipo' THEN e.escudo_url ELSE u.foto_perfil_url END AS
imagen,
       CASE WHEN p.tipo = 'equipo'
            THEN (SELECT COUNT(*)
                  FROM equipo_miembros m
                  WHERE m.equipo_id = p.referencia_id AND m.es_activo = 1)
            ELSE NULL END AS miembros,
       COALESCE(pos.puntos, 0) AS puntos
FROM participantes_torneo p
LEFT JOIN equipos e ON p.tipo = 'equipo' AND e.id = p.referencia_id
LEFT JOIN usuarios u ON p.tipo = 'usuario' AND u.id = p.referencia_id
LEFT JOIN torneo_posiciones pos ON pos.participante_id = p.id AND pos.torneo_id =
p.torneo_id
WHERE p.torneo_id = 1202
  AND p.estado = 'confirmado'
ORDER BY p.nombre;

-- Q53. Plantilla de los equipos inscriptos en un torneo.
-- Modelo: Torneo::obtenerParticipantesPublicos() (segunda consulta)
SELECT m.equipo_id,
       u.id,
       u.nombre_completo,
       u.foto_perfil_url,
       m.posicion
FROM equipo_miembros m
INNER JOIN usuarios u ON u.id = m.usuario_id
INNER JOIN participantes_torneo p
        ON p.tipo = 'equipo'
       AND p.referencia_id = m.equipo_id
       AND p.torneo_id = 1202
       AND p.estado = 'confirmado'
WHERE m.es_activo = 1
ORDER BY m.equipo_id, u.nombre_completo;

-- Q54. Tabla de posiciones publica de un torneo.
-- Modelo: Torneo::obtenerPosicionesPublicas()
SELECT p.nombre,
       COALESCE(pos.partidos_jugados, 0) AS partidos_jugados,
       COALESCE(pos.partidos_ganados, 0) AS partidos_ganados,
       COALESCE(pos.partidos_empatados, 0) AS partidos_empatados,
```

```sql
       COALESCE(pos.partidos_perdidos, 0) AS partidos_perdidos,
       COALESCE(pos.puntos_favor, 0) AS puntos_favor,
       COALESCE(pos.puntos_contra, 0) AS puntos_contra,
       COALESCE(pos.diferencia_goles, 0) AS diferencia_goles,
       COALESCE(pos.puntos, 0) AS puntos
FROM participantes_torneo p
LEFT JOIN torneo_posiciones pos ON pos.participante_id = p.id AND pos.torneo_id =
p.torneo_id
WHERE p.torneo_id = 1202
  AND p.estado = 'confirmado'
ORDER BY COALESCE(pos.puntos, 0) DESC,
         COALESCE(pos.diferencia_goles, 0) DESC,
         COALESCE(pos.puntos_favor, 0) DESC,
         p.nombre ASC;

-- Q55. Fixture publico de un torneo con escudos o fotos.
-- Modelo: Torneo::obtenerFixturePublico()
SELECT e.id,
       e.ronda,
       e.estado,
       e.fecha_hora_programada,
       e.cancha,
       e.resultado_local,
       e.resultado_visitante,
       e.participante_local_id,
       e.participante_visitante_id,
       e.participante_ganador_id,
       pl.nombre AS local_nombre,
       pv.nombre AS visitante_nombre,
       CASE WHEN pl.tipo = 'equipo' THEN el.escudo_url ELSE ul.foto_perfil_url END
AS local_imagen,
       CASE WHEN pv.tipo = 'equipo' THEN ev.escudo_url ELSE uv.foto_perfil_url END
AS visitante_imagen
FROM torneo_encuentros e
LEFT JOIN participantes_torneo pl ON pl.id = e.participante_local_id
LEFT JOIN participantes_torneo pv ON pv.id = e.participante_visitante_id
LEFT JOIN equipos el ON pl.tipo = 'equipo' AND el.id = pl.referencia_id
LEFT JOIN usuarios ul ON pl.tipo = 'usuario' AND ul.id = pl.referencia_id
LEFT JOIN equipos ev ON pv.tipo = 'equipo' AND ev.id = pv.referencia_id
LEFT JOIN usuarios uv ON pv.tipo = 'usuario' AND uv.id = pv.referencia_id
WHERE e.torneo_id = 1202
ORDER BY e.ronda ASC, e.id ASC;

-- Q56. Proximos enfrentamientos programados.
-- Modelo: Torneo::obtenerProximosEnfrentamientos()
SELECT e.id,
       e.torneo_id,
       e.fecha_hora_programada,
       e.cancha,
       t.stream_url,
       t.ubicacion,
       pl.nombre AS local_nombre,
       pv.nombre AS visitante_nombre,
       CASE WHEN pl.tipo = 'equipo' THEN el.escudo_url ELSE ul.foto_perfil_url END
```

```
AS local_imagen,
       CASE WHEN pv.tipo = 'equipo' THEN ev.escudo_url ELSE uv.foto_perfil_url END
AS visitante_imagen
FROM torneo_encuentros e
INNER JOIN torneos t ON t.id = e.torneo_id
INNER JOIN participantes_torneo pl ON pl.id = e.participante_local_id
INNER JOIN participantes_torneo pv ON pv.id = e.participante_visitante_id
LEFT JOIN equipos el ON pl.tipo = 'equipo' AND el.id = pl.referencia_id
LEFT JOIN usuarios ul ON pl.tipo = 'usuario' AND ul.id = pl.referencia_id
LEFT JOIN equipos ev ON pv.tipo = 'equipo' AND ev.id = pv.referencia_id
LEFT JOIN usuarios uv ON pv.tipo = 'usuario' AND uv.id = pv.referencia_id
WHERE e.estado = 'programado'
  AND e.fecha_hora_programada >= CURDATE()
  AND t.estado IN ('inscripciones_abiertas', 'en_curso')
ORDER BY e.fecha_hora_programada ASC
LIMIT 10;

-- Q57. Agenda publica: actividades y partidos en una sola linea de tiempo.
-- Modelo: Torneo::obtenerAgendaPublica()
SELECT fecha, hora, tipo, titulo, descripcion, estado, res_local, res_visitante
FROM (
    SELECT a.fecha,
           a.hora,
           a.tipo,
           a.titulo,
           a.descripcion,
           NULL AS estado,
           NULL AS res_local,
           NULL AS res_visitante
    FROM torneo_actividades a
    WHERE a.torneo_id = 1202
    UNION ALL
    SELECT DATE(e.fecha_hora_programada),
           TIME(e.fecha_hora_programada),
           'partido',
           CONCAT(COALESCE(pl.nombre, 'Por definir'), ' vs ', COALESCE(pv.nombre,
'Por definir')),
           NULL,
           e.estado,
           e.resultado_local,
           e.resultado_visitante
    FROM torneo_encuentros e
    LEFT JOIN participantes_torneo pl ON pl.id = e.participante_local_id
    LEFT JOIN participantes_torneo pv ON pv.id = e.participante_visitante_id
    WHERE e.torneo_id = 1202
      AND e.fecha_hora_programada IS NOT NULL
) agenda
ORDER BY fecha ASC, hora IS NULL, hora ASC;

-- Q58. Comentarios aprobados de un torneo.
-- Modelo: Torneo::obtenerComentariosPublicos()
SELECT c.id,
       c.torneo_id,
       c.usuario_id,
```

```
       c.nombre_autor,
       c.comentario,
       c.fecha_hora,
       u.nombre_completo AS nombre_usuario,
       u.foto_perfil_url
FROM torneo_comentarios c
LEFT JOIN usuarios u ON u.id = c.usuario_id
WHERE c.torneo_id = 1202
  AND c.aprobado = 1
ORDER BY c.fecha_hora DESC, c.id DESC
LIMIT 20;

-- Q59. Cantidad de comentarios aprobados de un torneo.
-- Modelo: Torneo::contarComentariosPublicos()
SELECT COUNT(*) AS comentarios_aprobados
FROM torneo_comentarios
WHERE torneo_id = 1202
  AND aprobado = 1;

-- Q60. Ultimos comentarios de la comunidad.
-- Modelo: Torneo::obtenerUltimosComentariosComunidad()
SELECT c.id,
       c.torneo_id,
       c.comentario,
       c.fecha_hora,
       COALESCE(NULLIF(u.nombre_completo, ''), c.nombre_autor) AS autor,
       u.foto_perfil_url AS foto,
       t.nombre AS torneo_nombre
FROM torneo_comentarios c
INNER JOIN torneos t ON t.id = c.torneo_id
LEFT JOIN usuarios u ON u.id = c.usuario_id
WHERE c.aprobado = 1
  AND t.estado NOT IN ('borrador', 'cancelado')
ORDER BY c.fecha_hora DESC, c.id DESC
LIMIT 10;

-- Q61. Podio de un torneo (tres primeras posiciones).
-- Modelo: Torneo::generarPodio()
WITH ranking AS (
    SELECT p.nombre,
           pos.puntos,
           pos.diferencia_goles,
           pos.puntos_favor,
           ROW_NUMBER() OVER (
               ORDER BY pos.puntos DESC, pos.diferencia_goles DESC,
                        pos.puntos_favor DESC, p.id ASC
           ) AS posicion
    FROM torneo_posiciones pos
    INNER JOIN participantes_torneo p ON p.id = pos.participante_id
    WHERE pos.torneo_id = 1200
)
SELECT posicion, nombre, puntos, diferencia_goles, puntos_favor
FROM ranking
WHERE posicion <= 3
```

```sql
ORDER BY posicion;

-- Q62. Campeon de un torneo segun el ultimo encuentro finalizado.
-- Modelo: Torneo::generarPodio()
SELECT e.ronda,
       ganador.nombre AS campeon
FROM torneo_encuentros e
INNER JOIN participantes_torneo ganador ON ganador.id = e.participante_ganador_id
WHERE e.torneo_id = 1200
  AND e.estado = 'finalizado'
ORDER BY e.ronda DESC, e.id DESC
LIMIT 1;

-- Q63. Configuracion guardada del podio.
-- Modelo: Torneo::obtenerPodio()
SELECT valor AS podio
FROM torneo_config
WHERE torneo_id = 1200
  AND clave = 'podio';

-- Q64. Historial de cambios de estado de un torneo.
-- Modelo: Torneo::obtenerHistorialOrganizador() y tabla torneo_cambio_estado
SELECT c.fecha_cambio,
       c.estado_anterior,
       c.estado_nuevo,
       u.nombre_completo AS usuario,
       c.motivo
FROM torneo_cambio_estado c
INNER JOIN usuarios u ON u.id = c.usuario_id
WHERE c.torneo_id = 1200
ORDER BY c.fecha_cambio, c.id;

-- Q65. Estadisticas del panel de un organizador.
-- Modelo: Torneo::obtenerEstadisticasOrganizador()
SELECT COALESCE(SUM(t.estado IN ('inscripciones_abiertas', 'en_curso')), 0) AS
torneos_activos,
       COALESCE(SUM(t.estado = 'finalizado'), 0) AS torneos_finalizados,
       COALESCE((SELECT COUNT(*)
                 FROM participantes_torneo p
                 INNER JOIN torneos t2 ON t2.id = p.torneo_id
                 WHERE t2.organizador_id = 1000
                   AND p.estado = 'pendiente'), 0) AS solicitudes_pendientes
FROM torneos t
WHERE t.organizador_id = 1000;

-- Q66. Torneos de un organizador con equipos confirmados.
-- Modelo: Torneo::obtenerTorneosPorOrganizador()
SELECT t.id,
       t.nombre,
       t.estado,
       t.formato,
       t.cupo_max_equipos,
       t.fecha_inicio,
       j.nombre AS nombre_juego,
```

```
       COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END) AS equipos_confirmados
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
WHERE t.organizador_id = 1000
GROUP BY t.id, t.nombre, t.estado, t.formato, t.cupo_max_equipos, t.fecha_inicio,
j.nombre
ORDER BY t.fecha_inicio IS NULL, t.fecha_inicio ASC, t.id DESC
LIMIT 20;

-- Q67. Todos los torneos de un organizador.
-- Modelo: Torneo::obtenerTodosPorOrganizador()
SELECT t.id,
       t.nombre,
       t.estado,
       t.formato,
       t.cupo_max_equipos,
       t.fecha_inicio,
       t.fecha_fin,
       j.nombre AS nombre_juego,
       j.categoria,
       COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END) AS equipos_confirmados
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
WHERE t.organizador_id = 1000
GROUP BY t.id, t.nombre, t.estado, t.formato, t.cupo_max_equipos,
         t.fecha_inicio, t.fecha_fin, j.nombre, j.categoria
ORDER BY t.creado_en DESC, t.id DESC;

-- Q68. Ultimos resultados de los torneos de un organizador.
-- Modelo: Torneo::obtenerResultadosOrganizador()
SELECT e.id,
       e.ronda,
       e.estado,
       e.resultado_local,
       e.resultado_visitante,
       e.ultima_modificacion,
       t.id AS torneo_id,
       t.nombre AS torneo_nombre,
       pl.nombre AS local_nombre,
       pv.nombre AS visitante_nombre
FROM torneo_encuentros e
INNER JOIN torneos t ON t.id = e.torneo_id
LEFT JOIN participantes_torneo pl ON pl.id = e.participante_local_id
LEFT JOIN participantes_torneo pv ON pv.id = e.participante_visitante_id
WHERE t.organizador_id = 1000
ORDER BY e.ultima_modificacion DESC, e.id DESC
LIMIT 50;

-- Q69. Calendario de un organizador: actividades y partidos.
-- Modelo: Torneo::obtenerCalendarioOrganizador()
SELECT fecha, hora, tipo, titulo, torneo_nombre, clase
FROM (
```

```
    SELECT a.fecha,
           a.hora,
           a.tipo,
           a.titulo,
           t.nombre AS torneo_nombre,
           'actividad' AS clase
    FROM torneo_actividades a
    INNER JOIN torneos t ON t.id = a.torneo_id
    WHERE t.organizador_id = 1000
    UNION ALL
    SELECT DATE(e.fecha_hora_programada) AS fecha,
           TIME(e.fecha_hora_programada) AS hora,
           'partido' AS tipo,
           CONCAT(COALESCE(pl.nombre, 'Pendiente'), ' vs ', COALESCE(pv.nombre,
'Pendiente')) AS titulo,
           t.nombre AS torneo_nombre,
           'partido' AS clase
    FROM torneo_encuentros e
    INNER JOIN torneos t ON t.id = e.torneo_id
    LEFT JOIN participantes_torneo pl ON pl.id = e.participante_local_id
    LEFT JOIN participantes_torneo pv ON pv.id = e.participante_visitante_id
    WHERE t.organizador_id = 1000
) calendario
ORDER BY fecha IS NULL, fecha ASC, hora IS NULL, hora ASC;

-- Q70. Partidos del calendario de un organizador.
-- Modelo: Torneo::obtenerPartidosCalendarioOrganizador()
SELECT e.id,
       e.ronda,
       e.resultado_local,
       e.resultado_visitante,
       DATE(e.fecha_hora_programada) AS fecha,
       TIME(e.fecha_hora_programada) AS hora,
       e.cancha AS sede,
       t.id AS torneo_id,
       t.nombre AS torneo_nombre,
       j.nombre AS juego_nombre,
       pl.nombre AS local_nombre,
       pv.nombre AS visitante_nombre
FROM torneo_encuentros e
INNER JOIN torneos t ON t.id = e.torneo_id
LEFT JOIN juegos j ON j.id = t.juego_id
LEFT JOIN participantes_torneo pl ON pl.id = e.participante_local_id
LEFT JOIN participantes_torneo pv ON pv.id = e.participante_visitante_id
WHERE t.organizador_id = 1000
ORDER BY e.fecha_hora_programada IS NULL, e.fecha_hora_programada ASC, e.id ASC;

-- Q71. Inscripciones de un torneo para el organizador.
-- Modelo: Torneo::obtenerInscripciones()
SELECT p.id,
       p.tipo,
       p.referencia_id,
       p.nombre,
       p.estado,
```

```
       p.fecha_inscripcion,
       t.cupo_max_equipos,
       CASE WHEN p.tipo = 'usuario'
            THEN (SELECT u.nombre_completo FROM usuarios u WHERE u.id =
p.referencia_id)
            ELSE (SELECT u.nombre_completo
                  FROM equipo_capitanes ec
                  INNER JOIN usuarios u ON u.id = ec.usuario_id
                  WHERE ec.equipo_id = p.referencia_id
                  ORDER BY ec.es_capitan_principal DESC
                  LIMIT 1)
       END AS responsable,
       (SELECT COUNT(*)
        FROM participantes_torneo confirmados
        WHERE confirmados.torneo_id = p.torneo_id
          AND confirmados.estado = 'confirmado') AS confirmados
FROM participantes_torneo p
INNER JOIN torneos t ON t.id = p.torneo_id
WHERE p.torneo_id = 1202
  AND t.organizador_id = 1002
ORDER BY FIELD(p.estado, 'pendiente', 'confirmado', 'rechazado'),
p.fecha_inscripcion ASC;

-- Q72. Actividades de un torneo para el organizador.
-- Modelo: Torneo::obtenerActividades()
SELECT a.id, a.titulo, a.descripcion, a.tipo, a.fecha, a.hora
FROM torneo_actividades a
INNER JOIN torneos t ON t.id = a.torneo_id
WHERE a.torneo_id = 1202
  AND t.organizador_id = 1002
ORDER BY a.fecha ASC, a.hora IS NULL, a.hora ASC;

-- Q73. Fixture de un torneo para el organizador.
-- Modelo: Torneo::obtenerFixture()
SELECT e.*,
       pl.nombre AS local_nombre,
       pv.nombre AS visitante_nombre
FROM torneo_encuentros e
INNER JOIN torneos t ON t.id = e.torneo_id
LEFT JOIN participantes_torneo pl ON pl.id = e.participante_local_id
LEFT JOIN participantes_torneo pv ON pv.id = e.participante_visitante_id
WHERE e.torneo_id = 1202
  AND t.organizador_id = 1002
ORDER BY e.ronda ASC, e.id ASC;

-- Q74. Posiciones de un torneo para el organizador.
-- Modelo: Torneo::obtenerPosiciones()
SELECT p.nombre, pos.*
FROM torneo_posiciones pos
INNER JOIN participantes_torneo p ON p.id = pos.participante_id
INNER JOIN torneos t ON t.id = pos.torneo_id
WHERE pos.torneo_id = 1202
  AND t.organizador_id = 1002
ORDER BY pos.puntos DESC, pos.diferencia_goles DESC, pos.puntos_favor DESC;
```

```sql
-- Q75. Cantidad de participantes confirmados de un torneo.
-- Modelo: Torneo::contarConfirmados()
SELECT COUNT(*) AS confirmados
FROM participantes_torneo
WHERE torneo_id = 1202
  AND estado = 'confirmado';

-- Q76. Verificar si un jugador o alguno de sus equipos ya esta inscripto.
-- Modelo: Torneo::obtenerInscripcionDelJugador()
SELECT *
FROM participantes_torneo
WHERE torneo_id = 1202
  AND (
      (tipo = 'usuario' AND referencia_id = 1010)
      OR (tipo = 'equipo' AND referencia_id IN (1100, 1101))
  )
  AND estado IN ('confirmado', 'pendiente')
ORDER BY FIELD(estado, 'confirmado', 'pendiente', 'rechazado', 'cancelado')
LIMIT 1;

-- Q77. Inscripciones de un jugador, directas o mediante sus equipos.
-- Modelo: Torneo::obtenerInscripcionesDelJugador()
SELECT p.id,
       p.tipo,
       p.nombre,
       p.estado,
       p.fecha_inscripcion,
       t.id AS torneo_id,
       t.nombre AS torneo_nombre,
       t.banner_url AS torneo_banner
FROM participantes_torneo p
INNER JOIN torneos t ON t.id = p.torneo_id
WHERE (p.tipo = 'usuario' AND p.referencia_id = 1010)
   OR (p.tipo = 'equipo' AND p.referencia_id IN (1100, 1101))
ORDER BY p.fecha_inscripcion DESC;

-- Q78. Miembros de un equipo que ya compiten con otro equipo en el mismo torneo.
-- Modelo: Torneo::obtenerMiembrosEnConflicto()
SELECT DISTINCT u.id,
       u.nombre_completo,
       e.nombre_equipo
FROM equipo_miembros em_nuevo
INNER JOIN usuarios u ON u.id = em_nuevo.usuario_id
INNER JOIN equipo_miembros em_existente
        ON em_existente.usuario_id = em_nuevo.usuario_id
       AND em_existente.es_activo = 1
INNER JOIN participantes_torneo pt
        ON pt.tipo = 'equipo'
       AND pt.referencia_id = em_existente.equipo_id
       AND pt.torneo_id = 1202
       AND pt.estado IN ('confirmado', 'pendiente')
INNER JOIN equipos e ON e.id = pt.referencia_id
WHERE em_nuevo.equipo_id = 1100
```

```sql
  AND em_nuevo.es_activo = 1
  AND pt.referencia_id <> 1100;

-- ============================================================================
-- NIVEL 5 - AGREGACIONES, RANKINGS Y REPORTES
-- ============================================================================

-- Q79. Cantidad de torneos por juego y estado.
-- Extra
SELECT j.id,
       j.nombre AS juego,
       COUNT(t.id) AS total_torneos,
       SUM(t.estado = 'borrador') AS borradores,
       SUM(t.estado = 'inscripciones_abiertas') AS abiertos,
       SUM(t.estado = 'en_curso') AS en_curso,
       SUM(t.estado = 'finalizado') AS finalizados,
       SUM(t.estado = 'cancelado') AS cancelados
FROM juegos j
LEFT JOIN torneos t ON t.juego_id = j.id
GROUP BY j.id, j.nombre
ORDER BY total_torneos DESC, j.nombre;

-- Q80. Organizadores con cantidad de torneos por estado (tabla pivote).
-- Extra (consulta pedida habitualmente en la defensa de Base de Datos)
SELECT u.id,
       u.nombre_completo,
       u.email,
       COALESCE(NULLIF(po.nombre_organizacion, ''), 'Organizador Independiente')
AS organizacion,
       COUNT(t.id) AS total_torneos,
       COALESCE(SUM(t.estado = 'inscripciones_abiertas'), 0) AS abiertos,
       COALESCE(SUM(t.estado = 'en_curso'), 0) AS en_curso,
       COALESCE(SUM(t.estado = 'finalizado'), 0) AS finalizados,
       COALESCE(SUM(t.estado = 'borrador'), 0) AS borradores,
       COALESCE(SUM(t.estado = 'cancelado'), 0) AS cancelados
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
LEFT JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
GROUP BY u.id, u.nombre_completo, u.email, po.nombre_organizacion
ORDER BY total_torneos DESC, u.nombre_completo;

-- Q81. Organizadores registrados con torneos abiertos.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(*) AS torneos_abiertos
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.estado = 'inscripciones_abiertas'
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY torneos_abiertos DESC, u.nombre_completo;
```

```sql
-- Q82. Organizadores registrados con torneos finalizados.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(*) AS torneos_finalizados
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.estado = 'finalizado'
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY torneos_finalizados DESC, u.nombre_completo;

-- Q83. Organizadores registrados con torneos en borrador.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(*) AS torneos_borrador
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.estado = 'borrador'
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY torneos_borrador DESC, u.nombre_completo;

-- Q84. Organizadores que tienen al menos un torneo abierto, uno finalizado
-- y uno en borrador al mismo tiempo.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       SUM(t.estado = 'inscripciones_abiertas') AS abiertos,
       SUM(t.estado = 'finalizado') AS finalizados,
       SUM(t.estado = 'borrador') AS borradores
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
GROUP BY u.id, u.nombre_completo, u.email
HAVING abiertos > 0
   AND finalizados > 0
   AND borradores > 0
ORDER BY u.nombre_completo;

-- Q85. Organizadores registrados que no tienen ningun torneo.
-- Extra
SELECT u.id, u.nombre_completo, u.email, u.fecha_registro
FROM usuarios u
LEFT JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.id IS NULL
ORDER BY u.nombre_completo;
```

```sql
-- Q86. Juegos con mas torneos organizados.
-- Modelo: Juego::obtenerJuegosDestacados()
SELECT j.id,
       j.nombre,
       j.categoria,
       j.imagen_url,
       COUNT(t.id) AS total_torneos
FROM juegos j
INNER JOIN torneos t ON t.juego_id = j.id
WHERE j.activo = 1
GROUP BY j.id, j.nombre, j.categoria, j.imagen_url
ORDER BY total_torneos DESC, j.nombre
LIMIT 10;

-- Q87. Top 10 de participantes por juego (equipos o jugadores).
-- Modelo: ResultadoGlobal::obtenerTop10PorJuego()
SELECT pt.nombre AS participante_nombre,
       pt.tipo AS participante_tipo,
       COUNT(DISTINCT tp.torneo_id) AS torneos_jugados,
       SUM(tp.partidos_jugados) AS partidos_jugados,
       SUM(tp.partidos_ganados) AS partidos_ganados,
       SUM(tp.puntos) AS puntos_totales
FROM torneo_posiciones tp
INNER JOIN participantes_torneo pt ON pt.id = tp.participante_id
INNER JOIN torneos t ON t.id = tp.torneo_id
WHERE t.estado IN ('finalizado', 'en_curso')
  AND t.juego_id = 1
GROUP BY pt.referencia_id, pt.tipo, pt.nombre
ORDER BY puntos_totales DESC, partidos_ganados DESC
LIMIT 10;

-- Q88. Ranking por juego y tipo de participante.
-- Modelo: ResultadoGlobal::obtenerRankingPorJuego()
SELECT pt.referencia_id AS id,
       COALESCE(NULLIF(MAX(pj.apodo_gamertag), ''), pt.nombre) AS nombre,
       COUNT(DISTINCT tp.torneo_id) AS torneos_jugados,
       SUM(tp.partidos_jugados) AS partidos_jugados,
       SUM(tp.partidos_ganados) AS partidos_ganados,
       SUM(tp.puntos) AS puntos_totales
FROM torneo_posiciones tp
INNER JOIN participantes_torneo pt ON pt.id = tp.participante_id
INNER JOIN torneos t ON t.id = tp.torneo_id
LEFT JOIN perfiles_jugadores pj
       ON pt.tipo = 'usuario'
      AND pj.usuario_id = pt.referencia_id
WHERE t.estado IN ('finalizado', 'en_curso')
  AND t.juego_id = 1
  AND pt.tipo = 'usuario'
GROUP BY pt.referencia_id, pt.nombre
ORDER BY puntos_totales DESC, partidos_ganados DESC, pt.nombre ASC
LIMIT 10;

-- Q89. Juegos que tienen resultados cargados.
-- Modelo: ResultadoGlobal::obtenerJuegosConResultados()
```

```sql
SELECT j.id, j.nombre, j.categoria
FROM juegos j
INNER JOIN torneos t ON t.juego_id = j.id
WHERE t.estado IN ('finalizado', 'en_curso')
GROUP BY j.id, j.nombre, j.categoria
ORDER BY j.nombre;

-- Q90. Juegos con tabla de posiciones generada.
-- Modelo: ResultadoGlobal::obtenerJuegosConRanking()
SELECT j.id, j.nombre, j.categoria
FROM juegos j
INNER JOIN torneos t ON t.juego_id = j.id
INNER JOIN torneo_posiciones tp ON tp.torneo_id = t.id
WHERE t.estado IN ('finalizado', 'en_curso')
GROUP BY j.id, j.nombre, j.categoria
ORDER BY j.nombre;

-- Q91. KPIs globales de competencia.
-- Modelo: ResultadoGlobal::obtenerKPIsGlobales()
SELECT SUM(tp.puntos) AS total_puntos_repartidos,
       SUM(tp.partidos_ganados) AS total_victorias_registradas,
       SUM(tp.partidos_jugados) AS total_partidos_jugados
FROM torneo_posiciones tp
INNER JOIN torneos t ON tp.torneo_id = t.id
WHERE t.estado IN ('finalizado', 'en_curso');

-- Q92. Estadisticas publicas generales.
-- Modelo: Torneo::obtenerEstadisticasPublicas()
SELECT (SELECT COUNT(*) FROM usuarios) AS usuarios,
       (SELECT COUNT(*) FROM torneo_encuentros WHERE estado = 'finalizado') AS
encuentros,
       (SELECT COUNT(*) FROM torneos WHERE estado NOT IN ('borrador',
'cancelado')) AS torneos,
       (SELECT COUNT(*) FROM equipos) AS equipos;

-- Q93. Top de jugadores por victorias.
-- Modelo: Torneo::obtenerTopJugadores()
SELECT u.id,
       COALESCE(NULLIF(MAX(pj.apodo_gamertag), ''), u.nombre_completo) AS nombre,
       u.foto_perfil_url AS imagen,
       SUM(pos.partidos_ganados) AS victorias
FROM participantes_torneo p
INNER JOIN torneos t ON t.id = p.torneo_id
INNER JOIN torneo_posiciones pos
        ON pos.participante_id = p.id
       AND pos.torneo_id = p.torneo_id
INNER JOIN usuarios u ON u.id = p.referencia_id
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
WHERE p.tipo = 'usuario'
  AND t.estado IN ('finalizado', 'en_curso')
GROUP BY u.id, u.nombre_completo, u.foto_perfil_url
HAVING victorias > 0
ORDER BY victorias DESC, nombre ASC
LIMIT 10;
```

```sql
-- Q94. Top de equipos por victorias.
-- Extra
SELECT p.nombre AS equipo,
       COUNT(DISTINCT pos.torneo_id) AS torneos,
       SUM(pos.partidos_jugados) AS partidos,
       SUM(pos.partidos_ganados) AS victorias,
       SUM(pos.puntos) AS puntos
FROM torneo_posiciones pos
INNER JOIN participantes_torneo p ON p.id = pos.participante_id
INNER JOIN torneos t ON t.id = pos.torneo_id
WHERE p.tipo = 'equipo'
  AND t.estado IN ('finalizado', 'en_curso')
GROUP BY p.referencia_id, p.nombre
ORDER BY victorias DESC, puntos DESC
LIMIT 10;

-- Q95. Ocupacion de cada torneo respecto de su cupo maximo.
-- Extra
SELECT t.id,
       t.nombre,
       t.estado,
       t.cupo_max_equipos,
       COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END) AS confirmados,
       ROUND(
           COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END)
           / t.cupo_max_equipos * 100, 1
       ) AS ocupacion_pct
FROM torneos t
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
GROUP BY t.id, t.nombre, t.estado, t.cupo_max_equipos
ORDER BY ocupacion_pct DESC, t.id;

-- Q96. Torneos con mas participantes confirmados.
-- Extra
SELECT t.id,
       t.nombre,
       t.estado,
       t.cupo_max_equipos,
       COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END) AS confirmados
FROM torneos t
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
GROUP BY t.id, t.nombre, t.estado, t.cupo_max_equipos
ORDER BY confirmados DESC, t.id;

-- Q97. Jugadores activos que no pertenecen a ningun equipo.
-- Extra
SELECT u.id, u.nombre_completo, u.email
FROM usuarios u
LEFT JOIN equipo_miembros em ON em.usuario_id = u.id AND em.es_activo = 1
WHERE u.rol_id = 3
  AND u.esta_activo = 1
  AND em.usuario_id IS NULL
ORDER BY u.nombre_completo;
```

```sql
-- Q98. Equipos activos que nunca participaron de un torneo.
-- Extra
SELECT e.id, e.nombre_equipo, e.fecha_creacion
FROM equipos e
LEFT JOIN participantes_torneo p ON p.tipo = 'equipo' AND p.referencia_id = e.id
WHERE e.activo = 1
  AND p.id IS NULL
ORDER BY e.nombre_equipo;

-- Q99. Juegos activos sin torneos.
-- Extra
SELECT j.id, j.nombre
FROM juegos j
LEFT JOIN torneos t ON t.juego_id = j.id
WHERE j.activo = 1
  AND t.id IS NULL
ORDER BY j.nombre;

-- Q100. Promedio de jugadores por equipo activo.
-- Extra
SELECT ROUND(AVG(total_miembros), 2) AS promedio_miembros
FROM (
    SELECT e.id, COUNT(em.usuario_id) AS total_miembros
    FROM equipos e
    LEFT JOIN equipo_miembros em ON em.equipo_id = e.id AND em.es_activo = 1
    WHERE e.activo = 1
    GROUP BY e.id
) por_equipo;

-- ============================================================================
-- NIVEL 6 - INTEGRIDAD, AUDITORIA Y ESTRUCTURA
-- ============================================================================

-- Q101. Un jugador anotado en dos equipos del mismo torneo.
-- Extra (regla de negocio: un jugador no puede competir con dos equipos en el
-- mismo torneo; esta consulta detecta conflictos)
SELECT t.id AS torneo_id,
       t.nombre AS torneo,
       u.id AS usuario_id,
       u.nombre_completo,
       COUNT(DISTINCT e.id) AS equipos_en_conflicto,
       GROUP_CONCAT(DISTINCT e.nombre_equipo ORDER BY e.nombre_equipo SEPARATOR '
| ') AS equipos
FROM usuarios u
INNER JOIN equipo_miembros em ON em.usuario_id = u.id AND em.es_activo = 1
INNER JOIN equipos e ON e.id = em.equipo_id AND e.activo = 1
INNER JOIN participantes_torneo p
        ON p.tipo = 'equipo'
       AND p.referencia_id = e.id
       AND p.estado IN ('confirmado', 'pendiente')
INNER JOIN torneos t
        ON t.id = p.torneo_id
       AND t.estado IN ('inscripciones_abiertas', 'en_curso')
```

```sql
GROUP BY t.id, t.nombre, u.id, u.nombre_completo
HAVING COUNT(DISTINCT e.id) > 1
ORDER BY t.id, u.nombre_completo;

-- Q102. Usuarios cuyo rol no coincide con sus perfiles especializados.
-- Extra (control de la herencia 1:1)
SELECT u.id,
       u.nombre_completo,
       u.rol_id,
       r.nombre_rol,
       pj.usuario_id AS tiene_perfil_jugador,
       po.usuario_id AS tiene_perfil_organizador
FROM usuarios u
INNER JOIN roles r ON r.id = u.rol_id
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE (u.rol_id = 3 AND pj.usuario_id IS NULL)
   OR (u.rol_id = 2 AND po.usuario_id IS NULL)
   OR (u.rol_id NOT IN (2, 3)
       AND (pj.usuario_id IS NOT NULL OR po.usuario_id IS NOT NULL))
ORDER BY u.id;

-- Q103. Participantes que apuntan a equipos inactivos o usuarios inactivos.
-- Extra
SELECT p.id,
       p.torneo_id,
       p.tipo,
       p.referencia_id,
       p.nombre,
       p.estado,
       CASE WHEN p.tipo = 'equipo' THEN e.activo ELSE u.esta_activo END AS
referencia_activa
FROM participantes_torneo p
LEFT JOIN equipos e ON p.tipo = 'equipo' AND e.id = p.referencia_id
LEFT JOIN usuarios u ON p.tipo = 'usuario' AND u.id = p.referencia_id
WHERE (p.tipo = 'equipo' AND (e.id IS NULL OR e.activo = 0))
   OR (p.tipo = 'usuario' AND (u.id IS NULL OR u.esta_activo = 0));

-- Q104. Torneos abiertos cuya fecha limite de inscripcion ya vencio.
-- Extra
SELECT id, nombre, fecha_limite_inscripcion, fecha_inicio
FROM torneos
WHERE estado = 'inscripciones_abiertas'
  AND fecha_limite_inscripcion IS NOT NULL
  AND fecha_limite_inscripcion < CURDATE();

-- Q105. Torneos finalizados sin podio guardado.
-- Extra
SELECT t.id, t.nombre, t.fecha_fin
FROM torneos t
LEFT JOIN torneo_config c
       ON c.torneo_id = t.id
      AND c.clave = 'podio'
WHERE t.estado = 'finalizado'
```

```sql
  AND c.torneo_id IS NULL;

-- Q106. Torneos finalizados con encuentros sin finalizar.
-- Extra
SELECT t.id,
       t.nombre,
       COUNT(*) AS encuentros_sin_finalizar
FROM torneos t
INNER JOIN torneo_encuentros e ON e.torneo_id = t.id
WHERE t.estado = 'finalizado'
  AND e.estado <> 'finalizado'
GROUP BY t.id, t.nombre;

-- Q107. Torneos en curso o finalizados sin fixture generado.
-- Extra
SELECT t.id, t.nombre, t.estado
FROM torneos t
LEFT JOIN torneo_encuentros e ON e.torneo_id = t.id
WHERE t.estado IN ('en_curso', 'finalizado')
  AND e.id IS NULL;

-- Q108. Resumen de integridad en una sola fila.
-- Extra
SELECT (SELECT COUNT(*) FROM usuarios WHERE rol_id = 4) AS usuarios_rol_publico,
       (SELECT COUNT(*) FROM equipos e
        LEFT JOIN equipo_miembros em
               ON em.equipo_id = e.id AND em.es_activo = 1
        WHERE em.usuario_id IS NULL) AS equipos_sin_miembros,
       (SELECT COUNT(*) FROM equipos e
        LEFT JOIN equipo_capitanes ec ON ec.equipo_id = e.id
        WHERE ec.equipo_id IS NULL) AS equipos_sin_capitan,
       (SELECT COUNT(*) FROM usuarios u
        LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
        WHERE u.rol_id = 3 AND pj.usuario_id IS NULL) AS jugadores_sin_perfil,
       (SELECT COUNT(*) FROM usuarios u
        LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
        WHERE u.rol_id = 2 AND po.usuario_id IS NULL) AS organizadores_sin_perfil,
       (SELECT COUNT(*) FROM torneos t
        LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
        WHERE t.estado IN ('en_curso', 'finalizado') AND p.id IS NULL) AS
torneos_sin_participantes;

-- Q109. Diccionario de datos: tablas y columnas de ascend.
-- Extra (muy util para explicar el modelo en la defensa)
SELECT table_name AS tabla,
       column_name AS columna,
       column_type AS tipo,
       is_nullable AS permite_nulo,
       column_key AS clave,
       column_default AS valor_defecto
FROM information_schema.columns
WHERE table_schema = 'ascend'
ORDER BY table_name, ordinal_position;
```

```sql
-- Q110. Claves foraneas del esquema.
-- Extra
SELECT table_name AS tabla_origen,
       column_name AS columna_origen,
       referenced_table_name AS tabla_destino,
       referenced_column_name AS columna_destino
FROM information_schema.key_column_usage
WHERE table_schema = 'ascend'
  AND referenced_table_name IS NOT NULL
ORDER BY table_name, column_name;

-- Q111. Tamano aproximado de cada tabla.
-- Extra
SELECT table_name AS tabla,
       table_rows AS filas_aproximadas,
       ROUND((data_length + index_length) / 1024 / 1024, 2) AS mb
FROM information_schema.tables
WHERE table_schema = 'ascend'
ORDER BY (data_length + index_length) DESC;

-- ============================================================================
-- ANEXO A - SENTENCIAS DE ESCRITURA DE LOS MODELOS (REFERENCIA)
-- ============================================================================
-- Estas sentencias NO forman parte de las consultas de defensa y NO se ejecutan
-- al correr el archivo: quedan documentadas para mostrar que la guia cubre las
-- operaciones reales y para explicar el flujo de escritura.
--
-- Equipo::crearEquipo()
-- INSERT INTO equipos (nombre_equipo, creado_por, juego_id, descripcion,
--                      ubicacion, anio_fundacion, escudo_url, banner_url,
--                      codigo_invitacion)
-- VALUES (:nombre, :creado_por, :juego_id, :descripcion, :ubicacion,
--         :anio_fundacion, :escudo_url, :banner_url, :codigo_invitacion);
--
-- Equipo::crearEquipoCompleto()
-- INSERT INTO equipo_capitanes
--     (equipo_id, usuario_id, asignado_por, es_capitan_principal)
-- VALUES (:equipo_id, :usuario_id, :asignado_por, 1);
--
-- Equipo::agregarMiembro()
-- INSERT INTO equipo_miembros (equipo_id, usuario_id, es_activo, fecha_union)
-- VALUES (:equipo_id, :usuario_id, 1, CURRENT_TIMESTAMP)
-- ON DUPLICATE KEY UPDATE es_activo = 1, fecha_union = CURRENT_TIMESTAMP;
--
-- Equipo::crearSolicitud()
-- INSERT INTO solicitudes_equipo
--     (equipo_id, usuario_id, iniciado_por, mensaje, estado)
-- VALUES (:equipo_id, :usuario_id, :iniciado_por, :mensaje, 'pendiente')
-- ON DUPLICATE KEY UPDATE estado = 'pendiente', fecha_solicitud =
CURRENT_TIMESTAMP;
--
-- Equipo::responderInvitacion() / responderSolicitudRecibida()
-- UPDATE solicitudes_equipo
-- SET estado = :estado, fecha_respuesta = NOW(), respondido_por = :respondido_por
```

```
-- WHERE id = :id;
--
-- Equipo::eliminarEquipo()
-- UPDATE equipos SET activo = 0 WHERE id = :id;
--
-- Torneo::inscribirse()
-- INSERT INTO participantes_torneo
--     (torneo_id, tipo, referencia_id, nombre, estado, fecha_inscripcion,
--      confirmado_por, fecha_confirmacion)
-- VALUES (:torneo_id, :tipo, :referencia_id, :nombre, 'pendiente',
--         CURRENT_TIMESTAMP, NULL, NULL)
-- ON DUPLICATE KEY UPDATE
--     estado = IF(estado IN ('rechazado', 'cancelado'), 'pendiente', estado),
--     nombre = VALUES(nombre);
--
-- Torneo::responderInscripcion()
-- UPDATE participantes_torneo
-- SET estado = :estado, confirmado_por = :confirmado_por,
--     fecha_confirmacion = CURRENT_TIMESTAMP
-- WHERE id = :id AND estado = 'pendiente';
--
-- Torneo::guardarResultado()
-- UPDATE torneo_encuentros
-- SET resultado_local = :local, resultado_visitante = :visitante,
--     participante_ganador_id = :ganador, estado = 'finalizado',
--     modificado_por_usuario_id = :usuario_id
-- WHERE id = :id AND estado <> 'finalizado';
--
-- Torneo::recalcularPosiciones()
-- UPDATE torneo_posiciones
-- SET partidos_jugados = :jugados, partidos_ganados = :ganados,
--     partidos_empatados = :empatados, partidos_perdidos = :perdidos,
--     puntos_favor = :favor, puntos_contra = :contra, puntos = :puntos
-- WHERE torneo_id = :torneo_id AND participante_id = :participante_id;
--
-- Jugador::actualizarPerfil() y Organizador::actualizarPerfil() usan
-- INSERT ... ON DUPLICATE KEY UPDATE sobre perfiles_jugadores y
-- perfiles_organizadores (upsert de la tabla hija 1:1).
--
-- Guardado::guardar() / quitar()
-- INSERT INTO guardados (usuario_id, tipo, referencia_id)
-- VALUES (:usuario_id, :tipo, :referencia_id);
-- DELETE FROM guardados
-- WHERE usuario_id = :usuario_id AND tipo = :tipo AND referencia_id =
:referencia_id;
--
-- Notificacion::marcarLeida()
-- UPDATE notificaciones
-- SET leido = 1, fecha_lectura = COALESCE(fecha_lectura, CURRENT_TIMESTAMP)
-- WHERE id = :id AND usuario_id = :usuario_id;
-- ============================================================================
```